



Faculté des Sciences et Techniques

Master 1 Ingénierie Mathématique et Data Science

Parcours : Ingénierie Mathématiques et Data Science

Mémoire de recherche

Thème

Résolution d'équations différentielles par réseaux de neurones artificiels

*De la méthode de Lagaris aux Physics-Informed Neural
Networks :
comparaison avec les méthodes numériques de pointe*

Présenté par :
kossi NOUMAGNO

Encadré par :
Prof. Cornel Marius MUREA
Responsable de formation :
Prof. Abdenacer MAKHLOUF

Année Universitaire 2025 – 2026

Table des matières

Remerciements	5
Introduction générale	6
1 Préliminaires mathématiques et apprentissage automatique	9
1.1 Équations différentielles ordinaires	9
1.1.1 Définitions et exemples	9
1.1.2 Fonctions lipschitziennes et théorème de Cauchy-Lipschitz	10
1.1.3 Interprétation et portée du théorème de Cauchy-Lipschitz	10
1.1.4 Vérification pratique de la condition de Lipschitz	11
1.1.5 Méthodes numériques de référence : schémas de Runge-Kutta	11
1.2 Réseaux de neurones artificiels	12
1.2.1 Perceptron et fonctions d'activation	12
1.2.2 Réseaux feedforward (perceptrons multicouches)	12
1.2.3 Fonction de perte et descente de gradient	12
1.2.4 Architectures profondes et optimiseurs modernes	13
1.3 Théorème d'approximation universelle	13
1.4 Lien entre analyse numérique et apprentissage automatique	14
1.4.1 Réseaux de neurones comme bases adaptatives	14
1.4.2 Interprétation comme méthode de collocation neuronale	14
2 Physics-Informed Neural Networks (PINNs)	15
2.1 Introduction	15
2.2 Historique	15
2.3 Formulation mathématique des PINNs	15
2.3.1 Problème générique	15
2.3.2 Approximation par un réseau de neurones et résidu physique	16
2.3.3 Fonction de perte composite : prise en compte des conditions initiales et aux limites	16

2.3.4	Problèmes directs et inverses	17
2.3.5	Illustration par un pseudo-code	17
2.4	Modèle en temps continu	17
2.5	Modèle en temps discret (Runge-Kutta implicites)	18
2.6	Limitation : le biais spectral et son remède	18
2.7	Lien avec la suite du mémoire	18
3	Étude expérimentale : le système de Lorenz	20
3.1	Introduction	20
3.2	Architecture des réseaux pour la résolution du système de Lorenz	21
3.2.1	Prétraitement des entrées et imposition de la condition initiale	21
3.2.2	PINN standard (sans Fourier Features)	21
3.2.3	PINN avec Fourier Features	21
3.2.4	Schéma récapitulatif	22
3.3	Calcul des résidus (perte ODE)	22
3.4	Fonction de perte et optimisation	23
3.5	Simulation 1 : Résolution directe (problème direct)	24
3.5.1	Objectif	24
3.5.2	Résultats quantitatifs	24
3.5.3	Visualisations	25
3.5.4	Interprétation	26
3.5.5	Conclusion partielle	26
3.6	Simulation 2 : Reconstruction à partir d'une seule variable (problème inverse)	26
3.6.1	Objectif	26
3.6.2	Protocole	27
3.6.3	Métrique d'erreur retenue	27
3.6.4	Résultats quantitatifs	27
3.6.5	Visualisations (cas Z observé)	28
3.6.6	Interprétation (cas Z observé)	29
3.6.7	Visualisations (cas Y observé)	29
3.6.8	Interprétation (cas Y observé)	30
3.6.9	Visualisations (cas X observé)	31
3.6.10	Interprétation (cas X observé)	32
3.6.11	Synthèse comparative des trois cas	32
3.6.12	Conclusion partielle – problème inverse	33
4	Synthèse, limites et perspectives	34
4.1	Bilan des résultats expérimentaux	34
4.2	Limites actuelles des PINNs	34
4.3	Perspectives	35
	Conclusion générale	36

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadrant, **Professeur Cornel Marius MUREA**, pour ses conseils avisés, sa disponibilité et la confiance qu'il m'a accordée tout au long de ce travail. Sa rigueur scientifique et sa passion pour les mathématiques appliquées m'ont été d'une grande inspiration.

Je remercie également le **Professeur Abdenacer MAKHLOUF**, responsable du Master Ingénierie Mathématique et Data Science, pour son accompagnement et les moyens mis à disposition.

Enfin, je suis reconnaissant envers l'ensemble des enseignants de la Faculté des Sciences et Techniques de l'UHA pour la qualité de leur formation.

Introduction générale

La modélisation des phénomènes physiques, biologiques ou technologiques conduit très souvent à des systèmes d'équations différentielles ordinaires (EDO) ou aux dérivées partielles (EDP). La résolution analytique de ces équations n'est possible que dans des cas très restreints, si bien que le recours à des méthodes numériques est incontournable. Les approches classiques différences finies, éléments finis, volumes finis, méthodes spectrales ont atteint une maturité remarquable et constituent des outils de référence pour l'ingénieur et le scientifique. Elles reposent toutefois sur une discrétisation spatiale et/ou temporelle qui peut devenir extrêmement coûteuse, voire prohibitive, lorsque la dimension du problème augmente (malédiction de la dimensionnalité) ou lorsque des échelles multiples sont en jeu. Par ailleurs, ces méthodes fournissent une solution discrète, souvent peu commode pour des calculs ultérieurs comme l'analyse de sensibilité ou l'optimisation.

Depuis plusieurs décennies, l'apprentissage automatique, et plus particulièrement les réseaux de neurones, se sont imposés comme des approximateurs universels de fonctions continues [2, 3]. Leur capacité à capturer des relations non linéaires complexes, leur caractère différentiable et leur facilité d'implémentation sur des architectures parallèles en font des candidats séduisants pour résoudre des équations différentielles.

La méthode de Lagaris : fondement des approches neuronales

Très tôt, des chercheurs ont proposé d'entraîner des réseaux de neurones pour qu'ils satisfassent, en un certain sens, une équation différentielle donnée. Cette idée a d'abord été concrétisée par Lagaris, Likas et Fotiadis [8] qui ont introduit une méthode de collocation neuronale. La solution d'essai y est décomposée en deux parties : l'une satisfait exactement les conditions aux limites (Dirichlet, Neumann ou mixtes) et ne contient aucun paramètre ajustable ; l'autre fait intervenir un réseau de neurones et s'annule sur les bords, préservant ainsi les conditions imposées. Le problème initial, avec contraintes, est alors ramené à un problème de minimisation sans contrainte où l'on cherche à annuler le résidu de l'équation en un nombre fini de points

de collocation. Cette approche fournit une solution analytique close, différentiable, et démontre des propriétés d'interpolation remarquables, tout en nécessitant très peu de paramètres. Elle pose ainsi les bases conceptuelles de la résolution neuronale d'EDO et d'EDP.

Les Physics-Informed Neural Networks : un cadre unifié

Cependant, la méthode de Lagaris présente deux limitations : la construction manuelle de la fonction satisfaisant les conditions aux limites peut s'avérer difficile pour des géométries complexes, et l'optimisation repose sur des techniques quasi-Newton classiques sans bénéficier des avancées modernes du *deep learning*. Ces verrous ont été levés par l'émergence des **Physics-Informed Neural Networks (PINNs)**, formalisées par Raissi, Perdikaris et Karniadakis [11]. Dans ce nouveau paradigme, la solution est représentée par un réseau de neurones profond, et les conditions aux limites ainsi que l'équation différentielle sont imposées de manière souple via une fonction de coût composite. La différentiation automatique permet de calculer avec une grande précision les résidus de l'équation, et l'optimisation est effectuée à l'aide d'algorithmes modernes (L-BFGS, Adam). Ce cadre unifié s'applique tant aux problèmes directs qu'aux problèmes inverses, et une variante en temps discret exploitant des schémas de Runge-Kutta implicites d'ordre arbitrairement élevé renforce encore son efficacité et sa stabilité.

Objectifs et problématique

L'**objectif de ce mémoire** est double. Dans un premier temps, nous revisiterons la méthode de Lagaris pour en comprendre les fondements, les techniques de construction des solutions d'essai et le calcul des gradients, qui constituent le socle historique des approches neuronales pour les équations différentielles. Cette étape nous permettra de poser les bases nécessaires à l'étude approfondie des PINNs, qui constituent le cœur de notre travail. Dans un second temps, nous confronterons les PINNs à une méthode numérique de référence, le solveur adaptatif **RK45** (Runge-Kutta d'ordre 4/5) [4], sur le **système chaotique de Lorenz** [9], paradigme de la sensibilité aux conditions initiales, qui constitue un banc d'essai exigeant pour toute méthode temporelle. Nous étudierons non seulement la résolution directe du système complet, mais aussi la **reconstruction de la dynamique** lorsqu'une seule variable (x , y ou z) est connue et que les autres doivent être inférées à partir de la connaissance des équations. Cette situation correspond à des problèmes inverses ou à des mesures partielles, fréquents en pratique.

La **problématique** centrale peut ainsi se formuler : les PINNs peuvent-elles rivaliser avec les meilleures méthodes numériques classiques en termes de précision et de vitesse, et sous quelles conditions deviennent-elles une alternative crédible pour la résolution de systèmes dynamiques non linéaires ?

Plan du mémoire

Le mémoire s'organise en quatre chapitres. Le **chapitre 1** rappelle les fondements mathématiques (EDO, systèmes dynamiques, théorème de Cauchy-Lipschitz, stabilité) et les concepts d'apprentissage automatique nécessaires (réseaux feedforward, théorème d'approximation universelle, descente de gradient). Le **chapitre 2** expose en détail les *Physics-Informed Neural Networks* : de leur genèse historique à leur formulation mathématique, en passant par la différentiation automatique et les modèles en temps continu et discret. Le **chapitre 3** est consacré à l'étude expérimentale sur le système de Lorenz : résolution directe, puis reconstruction à partir de chaque composante (x seul, y seul, z seul) avec analyse des erreurs et de la sensibilité. Enfin, le **chapitre 4** synthétise les résultats, discute les limites actuelles des PINNs et ouvre des perspectives sur les modèles hybrides et les opérateurs neuronaux.

Ce mémoire se positionne ainsi à l'intersection de l'analyse numérique et de l'apprentissage profond. Il vise à fournir une évaluation critique et rigoureuse d'un outil en plein essor, dont les retombées pour la simulation, le diagnostic et le contrôle des systèmes dynamiques sont considérables.

Préliminaires mathématiques et apprentissage automatique

1.1 Équations différentielles ordinaires

1.1.1 Définitions et exemples

Définition 1.1. Soit E un espace vectoriel normé (en pratique $E = \mathbb{R}^n$). Une **équation différentielle ordinaire** (EDO) d'ordre n est une équation de la forme

$$F(x, y, y', \dots, y^{(n)}) = 0, \quad (1.1)$$

où F est une fonction continue définie sur un ouvert $U \subset \mathbb{R} \times E^{n+1}$, appelé **domaine** de l'équation. L'ordre de l'équation est le plus grand ordre de dérivation qui y figure (ici n).

Définition 1.2 (Solution d'une EDO). Soit $I \subset \mathbb{R}$ un intervalle. Une fonction

$$y : I \rightarrow E$$

est dite **solution** de l'équation différentielle si elle vérifie les deux conditions suivantes :

- y est de classe C^n sur I (c'est-à-dire $y \in C^n(I, E)$);
- pour tout $x \in I$, on a

$$F(x, y(x), y'(x), \dots, y^{(n)}(x)) = 0. \quad (1.2)$$

Remarque 1.1. Résoudre une équation différentielle consiste à déterminer l'ensemble des fonctions y satisfaisant la relation ci-dessus. La solution générale dépend de n constantes arbitraires qui peuvent être fixées par des conditions initiales (problème de Cauchy) ou des conditions aux limites.

Exemple 1.1 (Oscillateur harmonique). Considérons l'équation différentielle linéaire du second ordre à coefficients constants :

$$y'' + y = 0.$$

Ses solutions sont les fonctions de la forme

$$y(x) = A \cos x + B \sin x,$$

où A et B sont des constantes réelles (ou complexes), déterminées par des conditions initiales (par exemple $y(0) = 1$, $y'(0) = 0$ donne $A = 1$, $B = 0$).

1.1.2 Fonctions lipschitziennes et théorème de Cauchy-Lipschitz

Définition 1.3 (Fonction lipschitzienne). Soit $(E, \|\cdot\|)$ un espace vectoriel normé et $f : I \times \Omega \rightarrow E$ une fonction définie sur un ouvert $I \times \Omega \subset \mathbb{R} \times E$. On dit que f est **lipschitzienne par rapport à sa deuxième variable** (ou simplement « lipschitzienne en y ») s'il existe une constante $L > 0$ telle que, pour tout $t \in I$ et pour tous $y_1, y_2 \in \Omega$,

$$\|f(t, y_1) - f(t, y_2)\| \leq L \|y_1 - y_2\|. \quad (1.3)$$

La constante L est appelée **constante de Lipschitz**. Si la propriété n'est vérifiée que localement (pour tout point il existe un voisinage où f est lipschitzienne), on dit que f est **localement lipschitzienne** par rapport à y .

Remarque 1.2. La condition de Lipschitz est plus forte que la simple continuité uniforme. Elle garantit un contrôle quantitatif de la variation de f et joue un rôle crucial dans la preuve du théorème d'existence et d'unicité.

Théorème 1.1 (Cauchy-Lipschitz (ou Picard-Lindelöf)). Soit $f : I \times \Omega \rightarrow E$ une fonction continue, et localement lipschitzienne par rapport à sa deuxième variable. Alors, pour toute condition initiale $(t_0, y_0) \in I \times \Omega$, le problème de Cauchy

$$\begin{cases} y'(t) = f(t, y(t)), \\ y(t_0) = y_0, \end{cases} \quad (1.4)$$

admet une **unique solution maximale** $y \in C^1(J, E)$ définie sur un intervalle ouvert $J \subset I$ contenant t_0 . De plus, cette solution dépend continûment des données initiales.

1.1.3 Interprétation et portée du théorème de Cauchy-Lipschitz

Remarque 1.3 (Bien-posé du problème de Cauchy). Le théorème de Cauchy-Lipschitz est un résultat fondamental en analyse des équations différentielles ordinaires. Il garantit que, sous une condition de Lipschitz sur le champ de vecteurs, le problème de Cauchy est **bien posé** au sens d'Hadamard :

- **Existence** d'une solution,
- **Unicité** de cette solution,
- **Dépendance continue** par rapport aux conditions initiales.

La démonstration repose sur la méthode des approximations successives de Picard, combinée au théorème du point fixe de Banach dans un espace de fonctions continues [4, 7].

1.1.4 Vérification pratique de la condition de Lipschitz

Le champ du système de Lorenz est polynomial, donc de classe C^1 et localement lipschitzien. Le théorème de Cauchy-Lipschitz s'applique donc, garantissant l'existence et l'unicité de la solution pour toute condition initiale. Cette propriété est essentielle pour la validation des méthodes numériques.

1.1.5 Méthodes numériques de référence : schémas de Runge-Kutta

La plupart des équations différentielles ordinaires non linéaires ne possèdent pas de solution analytique explicite. On recourt alors à des méthodes numériques permettant d'approcher la solution.

Parmi celles-ci, les méthodes de Runge-Kutta constituent une classe fondamentale de schémas d'intégration explicites. Elles reposent sur des évaluations successives du champ de vecteurs définissant l'équation différentielle, afin de construire une approximation locale de la solution.

Dans ce mémoire, nous utilisons comme méthode de référence la méthode adaptative **RK45 (Runge-Kutta-Fehlberg d'ordre 4(5))**. Cette méthode combine deux schémas d'ordres différents permettant d'estimer l'erreur locale et d'ajuster automatiquement le pas de temps. Elle est largement utilisée en pratique, notamment pour les systèmes dynamiques non linéaires et chaotiques.

Soit le problème de Cauchy :

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(t, \mathbf{x}), \quad \mathbf{x}(t_0) = \mathbf{x}_0, \quad (1.5)$$

la méthode RK45 construit donc, une approximation numérique $\mathbf{x}_n \approx \mathbf{x}(t_n)$ en adaptant dynamiquement le pas h afin de contrôler l'erreur locale.

Cette méthode est particulièrement adaptée aux systèmes sensibles aux conditions initiales, tels que le système de Lorenz, car elle permet un compromis efficace entre précision et coût de calcul.

Dans ce travail, RK45 est utilisée comme **référence numérique** pour :

- la validation des solutions obtenues par réseaux de neurones,
- la comparaison avec les Physics-Informed Neural Networks (PINNs),
- l'analyse de la stabilité et de la précision des méthodes apprenantes.

Ainsi, RK45 joue un rôle central dans le lien entre analyse numérique classique et méthodes modernes d'apprentissage automatique.

1.2 Réseaux de neurones artificiels

1.2.1 Perceptron et fonctions d'activation

Définition 1.4 (Perceptron). Un **perceptron** (ou *neurone formel*) est une unité de calcul élémentaire qui prend en entrée un vecteur $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, lui applique une somme pondérée par des poids $w_1, \dots, w_n$, ajoute un biais b et fait passer le résultat à travers une fonction d'activation $\sigma : \mathbb{R} \rightarrow \mathbb{R}$:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right). \quad (1.6)$$

Le vecteur des poids $\mathbf{w} = (w_1, \dots, w_n)$ et le biais b constituent les **paramètres** du perceptron [3].

Exemple 1.2 (Fonctions d'activation classiques). Les fonctions d'activation les plus courantes sont :

- Sigmoides : $\sigma(z) = \frac{1}{1 + e^{-z}}$,
- Tangente hyperbolique : $\sigma(z) = \tanh(z)$,
- Unité linéaire rectifiée (ReLU) : $\sigma(z) = \max(0, z)$.

La sigmoïde a été utilisée par Lagaris et al. [8], tandis que les PINNs de Raissi et al. [11] emploient souvent $\tanh$.

1.2.2 Réseaux feedforward (perceptrons multicouches)

Définition 1.5 (Perceptron multicouche — MLP). Un **réseau de neurones feedforward** (ou **perceptron multicouche**, MLP) est une composition de couches successives de perceptrons. La couche d'entrée reçoit le vecteur $\mathbf{x}$, chaque couche cachée calcule une transformation affine suivie d'une activation non linéaire, et la couche de sortie produit la prédiction. Pour un réseau à une seule couche cachée de H neurones, la sortie s'écrit

$$N(\mathbf{x}) = \sum_{i=1}^H v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i), \quad (1.7)$$

où $\mathbf{w}_i \in \mathbb{R}^n$ et $b_i \in \mathbb{R}$ sont les poids et biais de la couche cachée, et $v_i \in \mathbb{R}$ les poids de la couche de sortie [3].

1.2.3 Fonction de perte et descente de gradient

L'apprentissage supervisé consiste à ajuster les paramètres θ du réseau pour minimiser une **fonction de perte** $J(\theta)$ mesurant l'écart entre les prédictions et les valeurs cibles. La plus courante en régression est l'erreur quadratique moyenne (MSE) :

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left\| \hat{\mathbf{y}}_{\theta}(\mathbf{x}_i) - \mathbf{y}_i \right\|^2. \quad (1.8)$$

La **descente de gradient** est l'algorithme itératif

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} J(\theta_k), \quad (1.9)$$

où $\eta > 0$ est le **taux d'apprentissage**. Le gradient $\nabla_{\theta} J$ est calculé efficacement par l'algorithme de **rétropropagation** (backpropagation) qui applique la règle de la chaîne de manière récursive à travers les couches du réseau [3].

1.2.4 Architectures profondes et optimiseurs modernes

Les réseaux **profonds** (deep learning) possèdent plusieurs couches cachées, ce qui leur confère une plus grande capacité à extraire des motifs hiérarchiques. Leur entraînement nécessite des techniques avancées :

- **Optimiseur Adam** [6] : algorithme à pas adaptatif qui combine les idées du momentum et de RMSprop.
- **L-BFGS** [3] : méthode quasi-Newton qui approche la Hessienne et converge souvent plus rapidement sur les problèmes de taille modérée (utilisée par Raissi et al. [11]).
- **Régularisation** : *dropout* (désactivation aléatoire de neurones), *normalisation par lots* (batch normalization), et *weight decay* (pénalisation L_2).

1.3 Théorème d'approximation universelle

Théorème 1.2 (Cybenko, 1989 ; Hornik et al., 1989). *Soit $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ une fonction continue, bornée et non constante (par exemple la sigmoïde). Alors l'ensemble des réseaux à une couche cachée de la forme*

$$G(\mathbf{x}) = \sum_{i=1}^H \alpha_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) \quad (1.10)$$

*est **dense** dans l'espace des fonctions continues sur tout compact $K \subset \mathbb{R}^n$, muni de la norme uniforme. Autrement dit, pour toute fonction continue f et toute précision $\varepsilon > 0$, il existe un entier H et des paramètres $\alpha_i, \mathbf{w}_i, b_i$ tels que*

$$\sup_{\mathbf{x} \in K} |G(\mathbf{x}) - f(\mathbf{x})| < \varepsilon.$$

Ce résultat, dû à Cybenko [2] et généralisé par Hornik et al. [5], garantit l'existence d'un réseau suffisamment large approchant la solution d'une EDP/EDO avec une précision arbitraire.

1.4 Lien entre analyse numérique et apprentissage automatique

1.4.1 Réseaux de neurones comme bases adaptatives

L'analyse numérique classique construit des solutions approchées dans des espaces de fonctions fixés (polynômes par morceaux pour les éléments finis, séries de Fourier pour les méthodes spectrales). Les réseaux de neurones agissent comme une **base de fonctions non linéaires apprise** : la forme des fonctions de base $\sigma(\mathbf{w}_i^T \mathbf{x} + b_i)$ s'adapte automatiquement aux données et à la physique sous-jacente. Cette flexibilité leur permet de capturer des structures complexes avec relativement peu de paramètres [8, 11].

1.4.2 Interprétation comme méthode de collocation neuronale

La **méthode de collocation** consiste à imposer la satisfaction ponctuelle d'une équation différentielle en un nombre fini de points du domaine. Dans l'approche neuronale, on définit une **solution d'essai** paramétrée par un réseau $N(\mathbf{x}, \mathbf{p})$ (au sens de Lagaris) ou directement $u_\theta(t, x)$ (au sens des PINNs), puis on minimise le résidu de l'EDO/EDP évalué sur un ensemble de **points de collocation**. Ce point de vue unifie les travaux de Lagaris et al. [8] et les PINNs de Raissi et al. [11] : dans les deux cas, il s'agit d'entraîner un réseau pour qu'il satisfasse au mieux l'équation en un sens de moindres carrés.

Physics-Informed Neural Networks (PINNs)

2.1 Introduction

Les **Physics-Informed Neural Networks** (PINNs) constituent une avancée majeure à l'intersection du calcul scientifique et de l'apprentissage profond. Leur principe fondateur, énoncé par Raissi, Perdikaris et Karniadakis en 2019 [11], repose sur l'intégration de lois physiques exprimées sous forme d'EDO ou d'EDP directement dans la fonction de perte d'un réseau de neurones, au moyen de la **différentiation automatique**.

2.2 Historique

L'idée d'utiliser des réseaux de neurones pour résoudre des équations différentielles n'est pas nouvelle. Dès 1998, Lagaris, Likas et Fotiadis [8] proposaient une méthode de collocation neuronale où la solution d'essai satisfaisait automatiquement les conditions aux limites. Parallèlement, Psychogios et Ungar (1992) [10] introduisaient des modèles hybrides mêlant réseaux de neurones et équations physiques.

Cependant, ces approches restaient limitées par l'absence de différentiation automatique. L'avènement de cette technique dans les bibliothèques modernes (TensorFlow, PyTorch) [1] a permis à Raissi et al. de formuler le cadre général des PINNs, applicable aussi bien aux problèmes directs qu'inverses, en temps continu ou discret.

2.3 Formulation mathématique des PINNs

2.3.1 Problème générique

On considère une équation différentielle paramétrique de la forme générale

$$\mathcal{F}(t, x; u, u_t, u_x, \dots; \lambda) = 0, \quad t \in [0, T], \quad x \in \Omega \subset \mathbb{R}^d, \quad (2.1)$$

où $u(t, x)$ est la solution inconnue (latente) et λ un vecteur de paramètres physiques (coefficients de diffusion, vitesse, etc.). Le cas des EDO correspond à l'absence de variable spatiale x .

2.3.2 Approximation par un réseau de neurones et résidu physique

On approxime $u(t, x)$ par un réseau de neurones profond $u_\theta(t, x)$ de paramètres θ (poids et biais). On définit alors la **fonction résiduelle** (ou *réseau physique*) :

$$r_\theta(t, x) := \mathcal{F}(t, x; u_\theta, \partial_t u_\theta, \partial_x u_\theta, \dots; \lambda). \quad (2.2)$$

Toutes les dérivées partielles sont calculées exactement par **différentiation automatique**, sans erreur de discrétisation. Le réseau u_θ et le résidu r_θ partagent les mêmes paramètres θ ; l'opérateur $\mathcal{F}$ agit simplement comme une transformation des sorties du réseau.

2.3.3 Fonction de perte composite : prise en compte des conditions initiales et aux limites

L'apprentissage minimise une fonction de perte qui combine deux objectifs : la fidélité aux données disponibles (conditions initiales, conditions aux limites, éventuelles mesures internes) et le respect de l'équation différentielle. On écrit :

$$\mathcal{L}(\theta, \lambda) = \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{phys}}(\theta, \lambda), \quad (2.3)$$

avec :

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} |u_\theta(t_i, x_i) - u_i^{\text{obs}}|^2, \quad (2.4)$$

$$\mathcal{L}_{\text{phys}}(\theta, \lambda) = \frac{1}{N_{\text{colloc}}} \sum_{j=1}^{N_{\text{colloc}}} |r_\theta(t_j, x_j)|^2. \quad (2.5)$$

Les points $(t_i, x_i, u_i^{\text{obs}})$ correspondent à toutes les observations dont on dispose. En particulier, si le problème est bien posé, on impose :

- **Conditions initiales** : à $t = 0$, on connaît $u(0, x) = u_0(x)$. On dispose donc de N_0 points $(0, x_0^i, u_0^i)$.
- **Conditions aux limites** (Dirichlet, Neumann, périodiques) : pour $x \in \partial\Omega$, on connaît la valeur de u ou de sa dérivée normale.
- Éventuellement, des **mesures internes** à d'autres instants (dans le cas de problèmes inverses).

Toutes ces données sont regroupées dans $\mathcal{L}_{\text{data}}$. Ainsi, la perte sur la condition initiale s'écrit explicitement :

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_0} \sum_{i=1}^{N_0} \|u_\theta(0, x_0^i) - u_0^i\|^2,$$

et de même pour les conditions aux limites. Les points de collocation (t_j, x_j) sont échantillonnés aléatoirement à l'intérieur du domaine (y compris sur les bords) pour évaluer le résidu.

2.3.4 Problèmes directs et inverses

Le formalisme précédent permet de traiter deux classes de problèmes :

- **Problème direct (data-driven solution)** : les paramètres λ sont connus. On cherche à reconstruire la solution $u(t, x)$ sur tout le domaine en minimisant $\mathcal{L}(\theta)$. Le terme $\mathcal{L}_{\text{data}}$ contient uniquement les conditions initiales et aux limites.
- **Problème inverse (data-driven discovery)** : les paramètres λ sont inconnus. On les ajoute aux variables d'optimisation (θ, λ) . Le terme $\mathcal{L}_{\text{data}}$ peut inclure des mesures internes. Cette approche permet d'identifier les coefficients d'une EDO/EDP à partir d'observations, même bruitées.

L'article original [11] démontre la robustesse de cette identification en présence de bruit (jusqu'à 1% sur l'exemple de Navier-Stokes).

2.3.5 Illustration par un pseudo-code

La différentiation automatique est au cœur de la méthode. Avec PyTorch, le résidu pour l'équation de Burgers $u_t + uu_x - \nu u_{xx} = 0$ s'écrit simplement :

```
def residual(model, t, x, nu):
    u = model(t, x)
    u_t = torch.autograd.grad(u, t, grad_outputs=torch.ones_like(u),
                               create_graph=True)[0]
    u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),
                               create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u_x),
                                create_graph=True)[0]
    return u_t + u * u_x - nu * u_xx
```

On obtient ainsi le réseau physique sans aucun calcul de différences finies.

2.4 Modèle en temps continu

Dans le modèle en temps continu, le réseau prend en entrée les coordonnées (t, x) et apprend la solution sur l'intégralité du domaine spatio-temporel en une seule optimisation. Ce paradigme produit une solution continue et différentiable, sans maillage. Il est particulièrement performant pour les problèmes réguliers et nécessite peu de données (quelques dizaines de conditions initiales suffisent). L'article original illustre cette approche sur l'équation de Schrödinger non linéaire (section 3.1.1) et l'équation de Burgers (annexe A.1).

2.5 Modèle en temps discret (Runge-Kutta implicites)

Pour les problèmes d'évolution où un grand nombre de points de collocation temporels serait nécessaire, Raissi et al. proposent une variante discrète utilisant des schémas de **Runge-Kutta implicites à q étages**. On cherche à relier la solution à un instant t^n à la solution à l'instant $t^{n+1} = t^n + \Delta t$ par les équations :

$$u^{n+c_i} = u^n - \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \quad (2.6)$$

$$u^{n+1} = u^n - \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}], \quad (2.7)$$

où $u^{n+c_i}(x) = u(t^n + c_i \Delta t, x)$ et $\mathcal{N}$ est l'opérateur spatial (non linéaire). Les coefficients a_{ij}, b_j, c_j définissent le schéma (par exemple Gauss-Legendre).

Le réseau de neurones prend en entrée x et prédit simultanément les q étages u^{n+c_i} et la solution finale u^{n+1} :

$$[u^{n+c_1}(x), \dots, u^{n+c_q}(x), u^{n+1}(x)] = \text{NN}_\theta(x). \quad (2.8)$$

Grâce à la représentation implicite, le nombre d'étages q peut être très élevé (jusqu'à 500) sans augmentation significative du coût de calcul, car le réseau évalue tous les étages en parallèle. L'erreur de troncature temporelle est en $\mathcal{O}(\Delta t^{2q})$, ce qui permet, avec q suffisamment grand, d'atteindre la précision machine en un seul pas de temps, même pour des pas Δt très larges (exemple d'Allen-Cahn avec $\Delta t = 0.8$ et $q = 100$ dans l'article). De plus, les schémas de Gauss-Legendre sont A-stables, ce qui les rend idéaux pour les problèmes raides.

2.6 Limitation : le biais spectral et son remède

Une limite importante des PINNs standards réside dans leur **biais spectral** : les réseaux à activation lisse (tanh, sigmoïde) apprennent d'abord les basses fréquences, peinant à capturer les hautes fréquences ou les fronts raides. Ce phénomène est particulièrement gênant pour les systèmes dynamiques chaotiques (comme Lorenz) ou les équations de transport.

Une solution efficace, proposée par Tancik et al. [13], consiste à enrichir l'entrée du réseau par des **Fourier Features** : on remplace l'entrée t par un vecteur de fonctions sinusoïdales de fréquences multiples. Dans notre étude (chapitre 3), nous appliquons cette technique au système de Lorenz et montrons qu'elle réduit l'erreur d'un facteur important par rapport au PINN standard.

2.7 Lien avec la suite du mémoire

Le chapitre suivant applique les concepts de ce chapitre au système de Lorenz. Nous utiliserons le modèle en temps continu avec un réseau enrichi par des Fourier Features, et nous

évaluerons la capacité des PINNs à résoudre le problème direct (prédiction des trois variables) ainsi que des problèmes inverses (reconstruction à partir d'une seule composante mesurée). La fonction de perte inclura explicitement la condition initiale $\mathbf{u}(0) = \mathbf{u}_0$.

Étude expérimentale : le système de Lorenz

3.1 Introduction

Le système de Lorenz est un système dynamique chaotique tridimensionnel qui modélise de façon simplifiée la convection atmosphérique [9]. Il s'écrit :

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = x(\rho - z) - y, \\ \dot{z} = xy - \beta z, \end{cases} \quad (3.1)$$

Signification des variables

- x : intensité du mouvement de convection (analogue à une vitesse).
- y : différence de température horizontale entre les courants ascendants et descendants.
- z : écart du gradient vertical de température par rapport à un profil linéaire.

Ces interprétations proviennent du modèle de Rayleigh-Bénard et sont classiques dans la littérature [9, 12]. avec les paramètres canoniques $\sigma = 10$, $\rho = 28$, $\beta = 8/3$. Pour ces valeurs, la trajectoire est extrêmement sensible aux conditions initiales et évolue sur un attracteur fractal en forme de papillon [12]. L'intervalle temporel étudié est $t \in [0, 10]$ secondes, avec les conditions initiales $(x_0, y_0, z_0) = (1, 1, 1)$.

Dans ce chapitre, nous évaluons la capacité des PINNs à résoudre ce système, d'abord en problème direct (prédiction des trois composantes), puis en problème inverse (reconstruction de l'état complet à partir d'une seule variable mesurée). Nous comparons systématiquement deux architectures : un PINN standard et un PINN enrichi par des *Fourier Features*.

3.2 Architecture des réseaux pour la résolution du système de Lorenz

Afin de comparer la capacité des PINNs à résoudre le système de Lorenz, nous avons implémenté deux architectures de réseaux de neurones : un PINN standard avec une entrée purement temporelle, et un PINN enrichi par des *Fourier Features*. Les deux modèles partagent la même structure de perceptron multicouche (MLP) et la même méthode d'imposition exacte de la condition initiale.

3.2.1 Prétraitement des entrées et imposition de la condition initiale

L'intervalle temporel d'étude est $t \in [0, T]$ avec $T = 10$ secondes. Chaque temps t est d'abord normalisé dans $[-1, 1]$ par la transformation

$$\tau = \frac{2t}{T} - 1.$$

Le réseau prédit les **variables normalisées**

$$u_{\text{norm}}(\tau) = \left(\frac{x - \mu_x}{\sigma_x}, \frac{y - \mu_y}{\sigma_y}, \frac{z - \mu_z}{\sigma_z} \right),$$

où μ_x, μ_y, μ_z et $\sigma_x, \sigma_y, \sigma_z$ sont la moyenne et l'écart-type des variables d'état calculés sur la solution de référence RK45. Cette normalisation garantit que chaque composante contribue de façon comparable aux termes de la fonction de perte.

La condition initiale $\mathbf{u}(0) = \mathbf{u}_0$ est imposée **exactement** par la construction suivante du réseau :

$$u_{\text{norm}}(\tau) = \mathbf{u}_{0,\text{norm}} + \frac{\tau + 1}{2} \text{MLP}(\text{features}(\tau)),$$

où $\mathbf{u}_{0,\text{norm}} = (\mathbf{u}_0 - \boldsymbol{\mu})/\boldsymbol{\sigma}$. Pour $\tau = -1$ (soit $t = 0$), le terme $\frac{\tau+1}{2}$ s'annule, donc $u_{\text{norm}}(-1) = \mathbf{u}_{0,\text{norm}}$, ce qui satisfait rigoureusement la condition initiale sans terme de pénalité supplémentaire.

3.2.2 PINN standard (sans Fourier Features)

L'entrée du réseau est simplement τ (scalaire). On pose donc $\text{features}(\tau) = \tau$. Le MLP est un perceptron multicouche feed-forward à 4 couches cachées, chacune comportant 256 neurones et utilisant la fonction d'activation tanh. La couche de sortie est linéaire et produit trois valeurs (les trois composantes normalisées). Le nombre total de paramètres est de 132 867.

3.2.3 PINN avec Fourier Features

Pour pallier le **biais spectral** des réseaux à activation lisse, l'entrée est enrichie par des fonctions sinusoïdales de fréquences multiples. On choisit $K = 6$ fréquences $f_k \in \{1, 2, 4, 8, 16, 32\}$.

Pour chaque fréquence, on ajoute deux caractéristiques :

$$\sin(\pi f_k(\tau + 1)), \quad \cos(\pi f_k(\tau + 1)).$$

Le vecteur d'entrée final a donc une dimension $1 + 2K = 13$:

$$\text{features}(\tau) = \left[\tau, \sin(\pi f_1(\tau + 1)), \cos(\pi f_1(\tau + 1)), \dots, \sin(\pi f_6(\tau + 1)), \cos(\pi f_6(\tau + 1)) \right]^\top.$$

Ce vecteur est ensuite injecté dans un MLP de même architecture que le PINN standard (4 couches cachées, 256 neurones, tanh, sortie linéaire). Le nombre total de paramètres est de 135 939 (augmentation de 2,3%).

3.2.4 Schéma récapitulatif

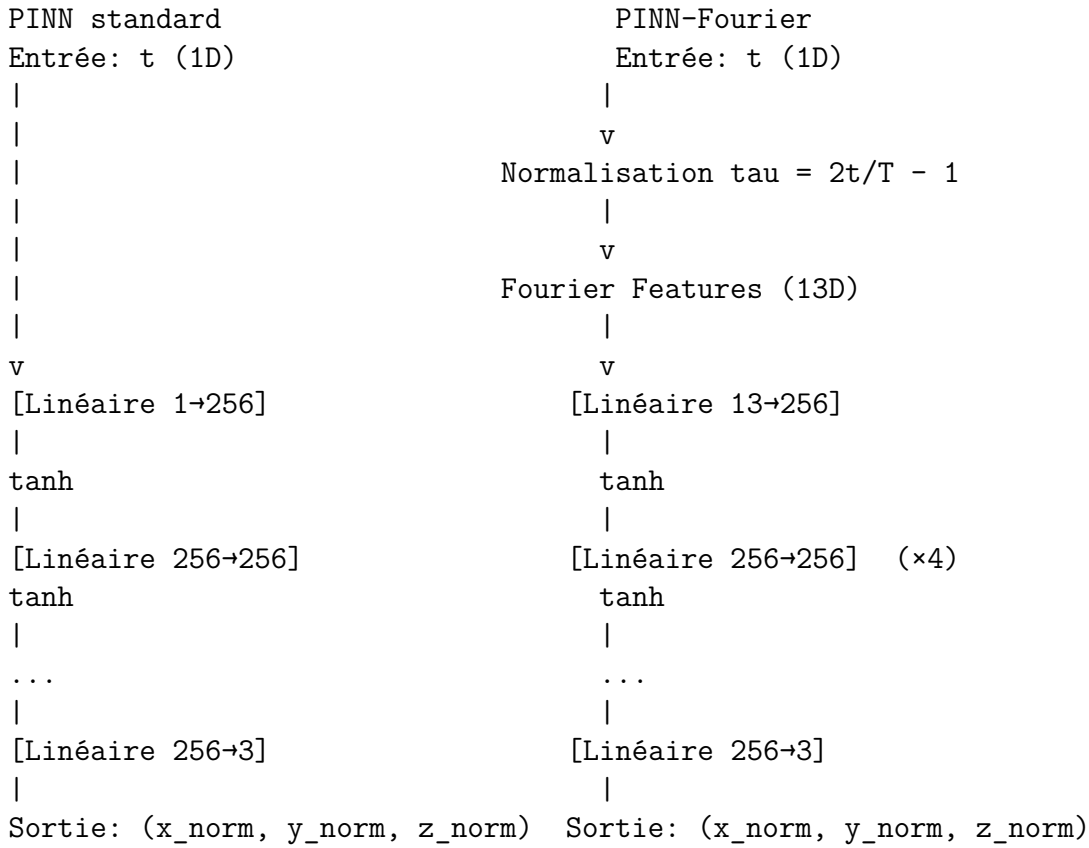


FIGURE 3.1 – Architectures comparées du PINN standard et du PINN avec Fourier Features.

3.3 Calcul des résidus (perte ODE)

Pour les deux modèles, la fonction de coût contient un terme de pénalité qui impose la satisfaction des équations de Lorenz. Les dérivées temporelles $\dot{x}, \dot{y}, \dot{z}$ sont calculées par différentiation automatique (fonction `torch.autograd.grad`) à partir des sorties du réseau. On définit

les résidus :

$$f_1(t) = \dot{x} - \sigma(y - x), \quad f_2(t) = \dot{y} - (x(\rho - z) - y), \quad f_3(t) = \dot{z} - (xy - \beta z),$$

avec $\sigma = 10$, $\rho = 28$, $\beta = 8/3$.

Afin d'équilibrer l'influence des trois composantes (d'amplitudes très différentes), chaque résidu est divisé par un facteur d'échelle dépendant de la composante :

$$\text{fac}_x = \frac{\sigma_x}{T}, \quad \text{fac}_y = \frac{\sigma_y}{T}, \quad \text{fac}_z = \frac{\sigma_z}{T},$$

où $\sigma_x, \sigma_y, \sigma_z$ sont les écarts-types des variables de référence RK45, et $T = 10$ s. La perte ODE est alors la moyenne quadratique des résidus normalisés sur l'ensemble des points de collocation :

$$\mathcal{L}_{\text{ODE}} = \frac{1}{3N_{\text{colloc}}} \sum_{j=1}^{N_{\text{colloc}}} \left[\left(\frac{f_1(t_j)}{\text{fac}_x} \right)^2 + \left(\frac{f_2(t_j)}{\text{fac}_y} \right)^2 + \left(\frac{f_3(t_j)}{\text{fac}_z} \right)^2 \right].$$

3.4 Fonction de perte et optimisation

La fonction de perte totale combine la perte de données (supervisée) et la perte ODE :

$$\mathcal{L} = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{ODE}} \mathcal{L}_{\text{ODE}}.$$

Perte de données On dispose de $N_{\text{data}} = 500$ points issus de la solution RK45, échantillonnés uniformément sur $[0, 10]$. Les variables sont normalisées comme décrit précédemment. La perte de données est la somme des carrés des écarts entre les prédictions normalisées et les valeurs de référence, sur les composantes observées (dans le problème direct, toutes les trois sont observées) :

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \left\| u_{\text{norm},\theta}(t_i) - u_{\text{norm,ref}}(t_i) \right\|^2.$$

Optimisation par curriculum L'entraînement se déroule en trois phases :

1. **Pré-apprentissage supervisé** (5000 époques) : seules les données sont utilisées, avec $\lambda_{\text{data}} = 20$ et $\lambda_{\text{ODE}} = 0,02$. L'optimiseur est Adam avec un taux d'apprentissage initial $\eta = 10^{-3}$. Cette phase permet au réseau d'apprendre une approximation grossière de la trajectoire avant d'introduire la contrainte physique.
2. **Phase PINN avec curriculum** (10 000 époques) : on utilise $\lambda_{\text{data}} = 20$ et λ_{ODE} est augmenté progressivement. Un *curriculum* linéaire fait croître λ_{ODE} de 10^{-5} à 0,05 pendant les 5000 premières époques, puis le maintient à 0,05 pour les 5000 époques restantes. L'optimiseur Adam utilise un taux d'apprentissage $\eta = 5 \times 10^{-4}$. Un scheduler réduit le taux de moitié toutes les 2000 époques.
3. **Raffinement avec L-BFGS** (300 itérations) : après les phases Adam, on affine les paramètres avec l'algorithme L-BFGS (recherche linéaire de Wolfe) en fixant $\lambda_{\text{data}} = 10$

et $\lambda_{\text{ODE}} = 0,05$.

Les points de collocation pour la perte ODE sont au nombre de $N_{\text{colloc}} = 1000$, distincts des points de données (mais également répartis uniformément sur l'intervalle). La condition initiale étant imposée exactement par construction du réseau, aucun terme de pénalité supplémentaire n'est nécessaire.

3.5 Simulation 1 : Résolution directe (problème direct)

3.5.1 Objectif

Apprendre la solution complète $(x(t), y(t), z(t))$ à partir des équations de Lorenz et de 500 points de référence issus du solveur RK45. Il s'agit de la variante supervisée + physique ($\lambda_{\text{data}} = 1$).

3.5.2 Résultats quantitatifs

Le tableau 3.1 présente les erreurs relatives L2 par composante et l'erreur globale pour les deux architectures.

TABLE 3.1 – Métriques d'erreur pour la résolution directe du système de Lorenz (500 points de données, 1000 points de collocation).

Modèle	Erreur relative L2			Global L2
	X	Y	Z	
PINN standard	0.1532	0.2169	0.0764	0.1052
PINN-Fourier	0.0702	0.1473	0.0399	0.0609

Le PINN-Fourier réduit l'erreur globale d'un facteur 1,7 par rapport au PINN standard. L'amélioration est particulièrement marquée sur la composante Z .

3.5.3 Visualisations

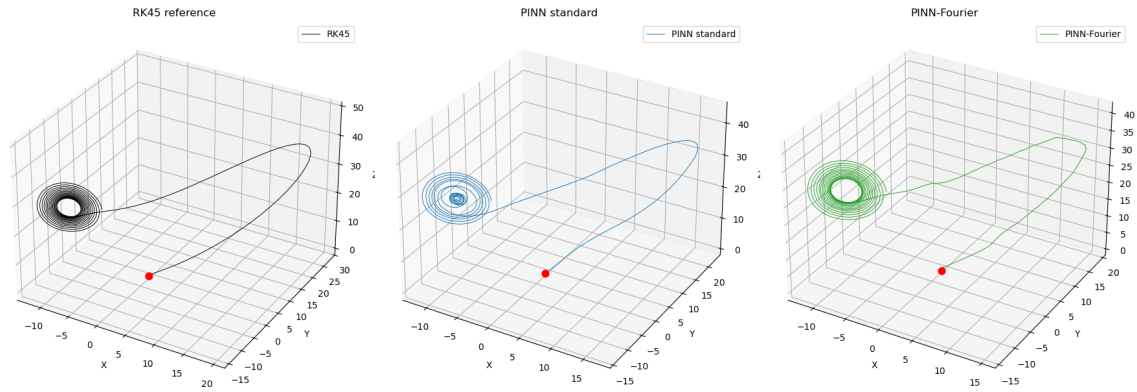


FIGURE 3.2 – Attracteurs de Lorenz reconstruits par RK45 (référence), PINN standard et PINN-Fourier. L'attracteur du PINN-Fourier est quasi superposable à la référence, tandis que celui du PINN standard présente des déformations visibles.

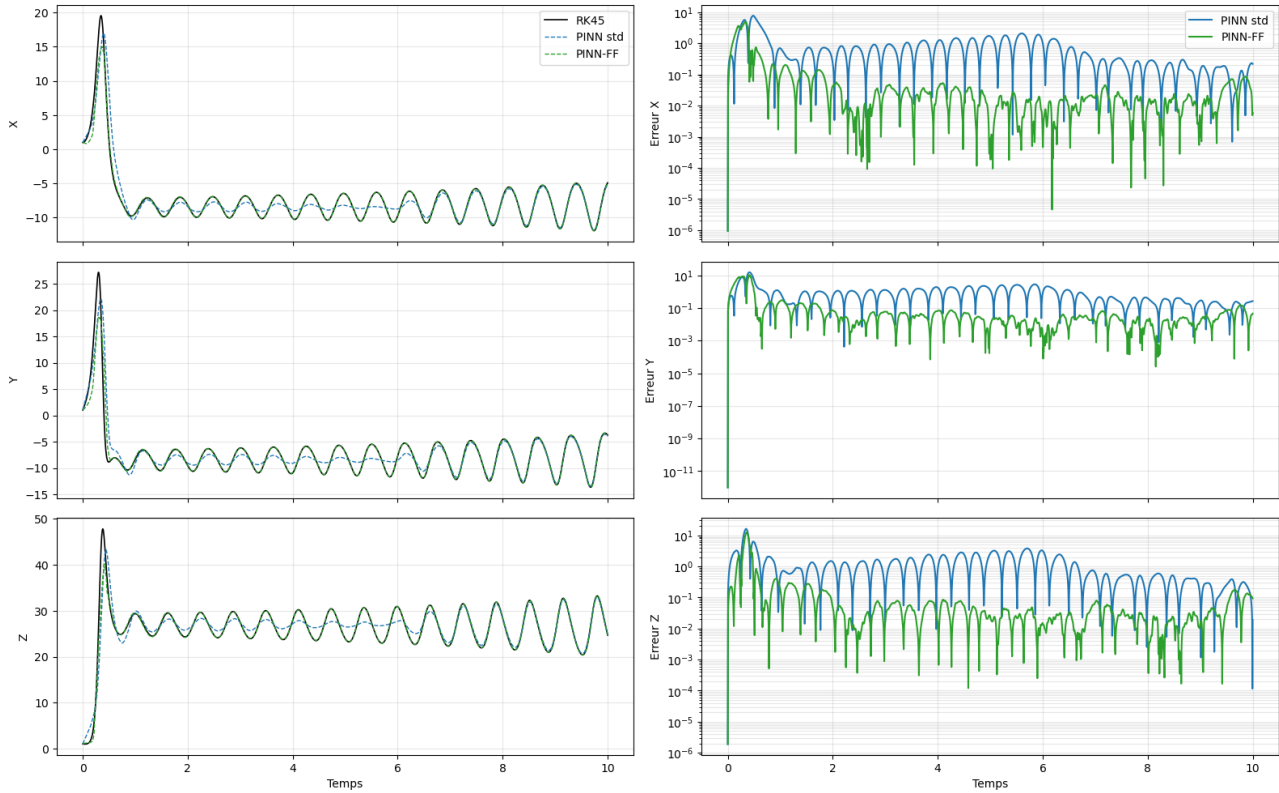


FIGURE 3.3 – Évolution temporelle des composantes x , y , z (gauche) et des erreurs absolues (droite, échelle logarithmique). Le PINN-Fourier maintient une erreur faible sur tout l'intervalle, alors que celle du PINN standard s'explode par la suite.

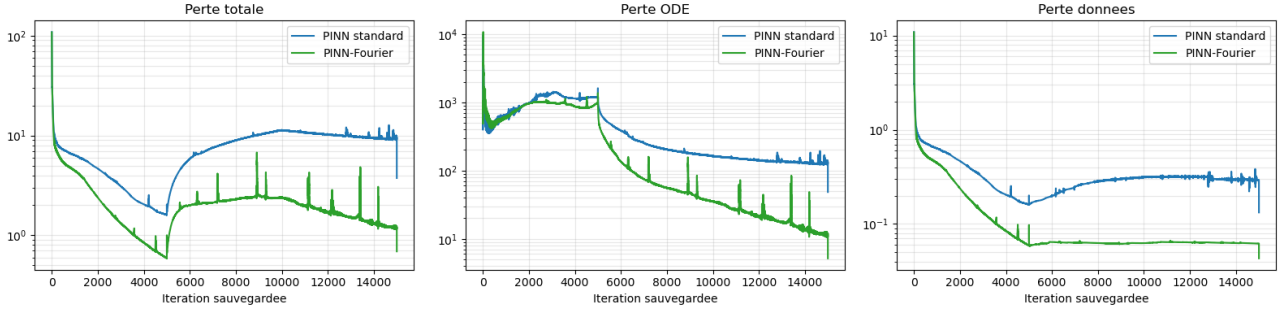


FIGURE 3.4 – Évolution des pertes au cours de l'entraînement. La perte ODE du PINN-Fourier descend nettement plus bas, témoignant d'une meilleure satisfaction des équations physiques.

3.5.4 Interprétation

La figure 3.2 confirme visuellement la supériorité du PINN-Fourier : l'attracteur reconstruit est quasi superposable à la référence RK45, alors que celui du PINN standard présente des déformations notables, en particulier dans la zone des deux lobes.

L'analyse temporelle (figure 3.3) montre que l'erreur du PINN standard augmente rapidement après un certain temps (jusqu'à dépasser 1), tandis que celle du PINN-Fourier reste inférieure à l'erreur du PINN standard sur tout l'intervalle. Ce comportement illustre la capacité des *Fourier Features* à stabiliser l'apprentissage des hautes fréquences, essentielles pour les régimes chaotiques.

Enfin, les courbes d'apprentissage (figure 3.4) indiquent que le résidu ODE du PINN-Fourier descend plus bas (en dessous de 10^{-1}), attestant d'une meilleure satisfaction des équations de Lorenz. La perte sur les données reste faible pour les deux modèles grâce au pré-apprentissage supervisé initial.

3.5.5 Conclusion partielle

Pour la résolution directe, le PINN enrichi par des *Fourier Features* surpasse nettement le PINN standard, tant en précision qu'en stabilité. Ce gain justifie l'utilisation systématique de cet encodage fréquentiel pour les systèmes dynamiques chaotiques.

3.6 Simulation 2 : Reconstruction à partir d'une seule variable (problème inverse)

3.6.1 Objectif

Dans de nombreuses applications pratiques, seule une partie des variables d'état est mesurable (par exemple, un seul capteur disponible). On souhaite reconstruire la dynamique complète $(x(t), y(t), z(t))$ à partir de l'observation d'une seule composante. Trois cas sont étudiés : observation de $z(t)$ seulement, observation de $y(t)$ seulement, observation de $x(t)$ seulement.

3.6.2 Protocole

Pour chaque cas, la fonction de perte est modifiée : le terme de données ne porte que sur la variable observée (500 points issus de la référence RK45). Les deux autres variables ne sont contraintes que par la physique (perte ODE) et par la condition initiale. Les hyperparamètres (architecture des réseaux, 500 points de collocation, curriculum d'apprentissage) sont strictement identiques à ceux de la simulation directe.

3.6.3 Métrique d'erreur retenue

L'erreur relative L2 ($\text{RelL2} = \frac{\|\text{pred} - \text{ref}\|}{\|\text{ref}\|}$) est privilégiée car elle normalise les différences par l'amplitude de la référence. Elle permet de comparer des composantes aux dynamiques d'amplitudes très différentes (par exemple x et z dans le système de Lorenz) et de quantifier équitablement la performance entre problèmes directs et inverses. Les métriques absolues (MAE, RMSE) sont également calculées mais ne sont pas rapportées dans les tableaux principaux ; elles peuvent être consultées en annexe.

3.6.4 Résultats quantitatifs

Cas Z observé

Le tableau 3.2 présente les erreurs relatives L2 par composante et l'erreur globale pour les deux architectures.

TABLE 3.2 – Métriques d'erreur pour la reconstruction à partir de la seule composante Z (500 points de données sur Z , 500 points de collocation).

Modèle	Erreur relative L2			Global L2
	X (cachée)	Y (cachée)	Z (observée)	
PINN standard	0.46538	0.52117	0.12317	0.23516
PINN-Fourier	0.45483	0.48049	0.06972	0.20558

Cas Y observé

TABLE 3.3 – Métriques d'erreur pour la reconstruction à partir de la seule composante Y .

Modèle	Erreur relative L2			Global L2
	X (cachée)	Y (observée)	Z (cachée)	
PINN standard	0.31489	0.34593	0.23090	0.25135
PINN-Fourier	0.27416	0.22523	0.18369	0.19680

Cas X observé

TABLE 3.4 – Métriques d'erreur pour la reconstruction à partir de la seule composante X .

Modèle	Erreur relative L2			Global L2
	X (observée)	Y (cachée)	Z (cachée)	
PINN standard	0.29482	0.40692	0.25705	0.27729
PINN-Fourier	0.18154	0.31354	0.20467	0.21517

3.6.5 Visualisations (cas Z observé)

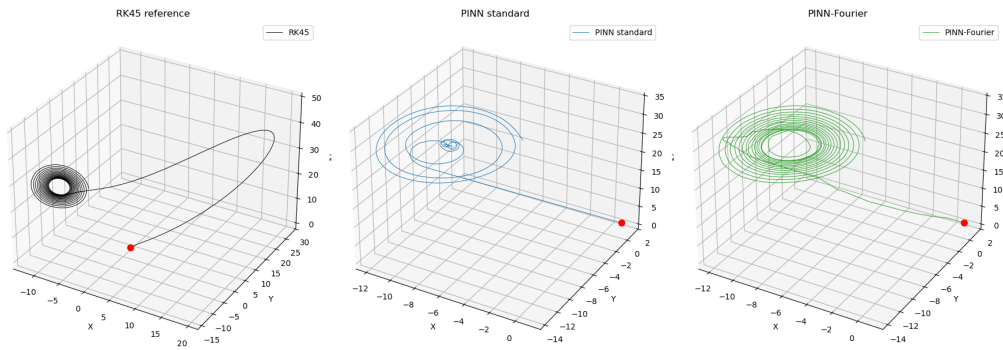


FIGURE 3.5 – Attracteurs de Lorenz reconstruits par RK45 (référence), PINN standard et PINN-Fourier en n'observant que la variable Z . Le PINN-Fourier parvient à restituer la structure globale en papillon, tandis que l'attracteur du PINN standard est déformé.

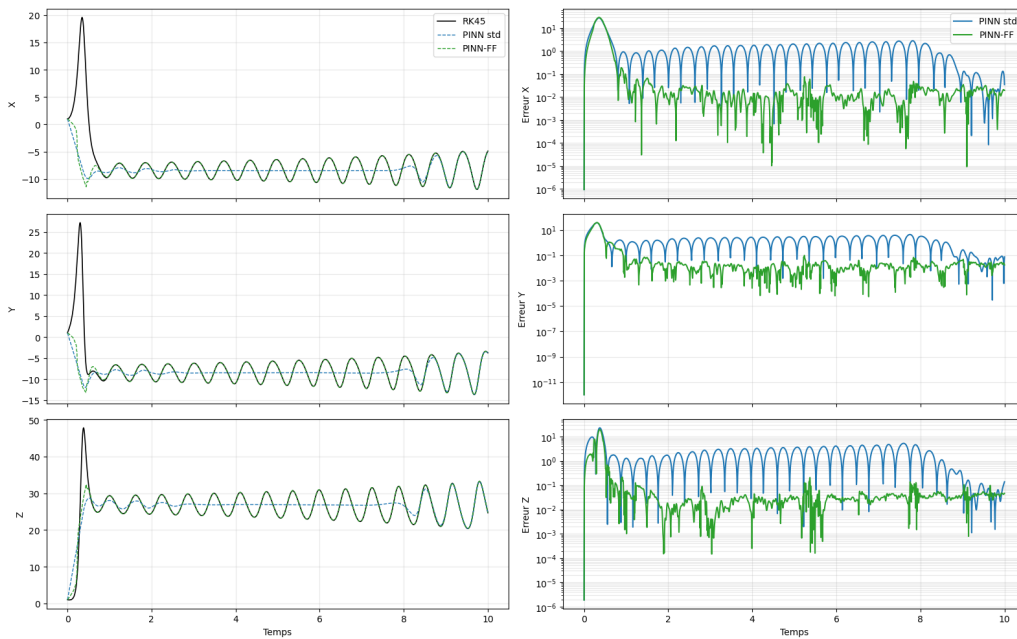


FIGURE 3.6 – Évolution temporelle (gauche) et erreurs absolues (droite, échelle logarithmique) pour les trois composantes. La composante observée Z est correctement reconstruite par les deux modèles, mais les variables cachées X et Y divergent rapidement. Le PINN-Fourier réduit l'erreur sur Z d'un facteur deux.

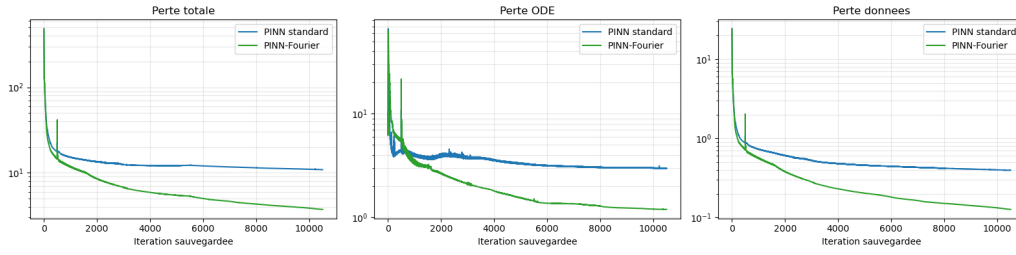


FIGURE 3.7 – Évolution des pertes au cours de l'entraînement. La perte ODE du PINN-Fourier descend plus bas (environ 10^{-1}), indiquant une meilleure satisfaction des équations de Lorenz.

3.6.6 Interprétation (cas Z observé)

La figure 3.5 montre que le PINN-Fourier restitue la topologie globale de l'attracteur (les deux lobes), alors que le PINN standard produit une structure déformée. L'analyse temporelle (figure 3.6) confirme que les erreurs sur X et Y explosent après $t \approx 2-3$ secondes, tandis que l'erreur sur Z reste modérée. Ce comportement est typique d'un système chaotique faiblement observable : les petites incertitudes sur l'état initial ou sur la dynamique sont exponentiellement amplifiées. Les courbes d'apprentissage (figure 3.7) indiquent que le PINN-Fourier satisfait mieux les équations différentielles (perte ODE plus basse), ce qui explique sa meilleure reconstruction qualitative de l'attracteur. Cependant, même une perte ODE faible ne suffit pas à rendre X et Y précis, car l'information contenue dans Z est intrinsèquement insuffisante pour lever les dégénérescences.

3.6.7 Visualisations (cas Y observé)

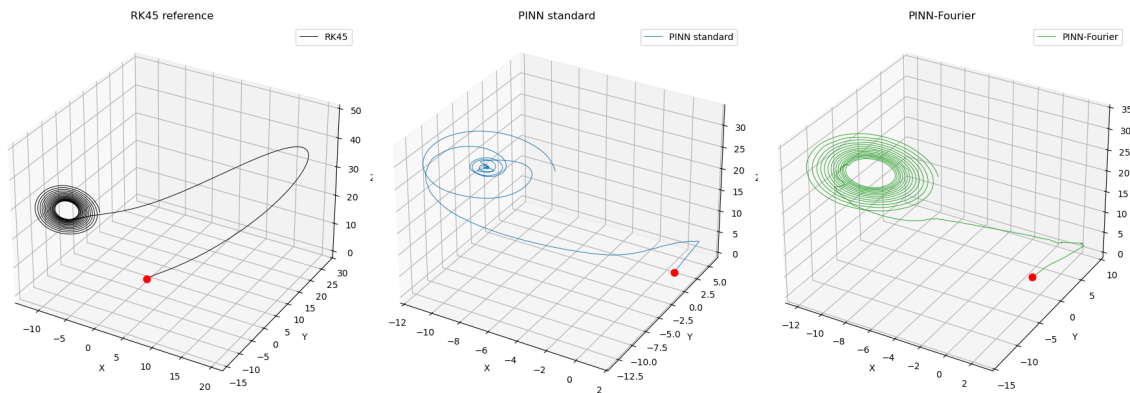


FIGURE 3.8 – Comparaison des trajectoires 3D et reconstruction de l'attracteur en n'observant que la variable Y . La référence RK45 montre une structure spirale stable convergeant vers un cycle limite. Le PINN standard s'effondre prématurément vers le centre et ne capture pas les oscillations. Le PINN-Fourier suit la spirale avec une fidélité remarquable, respectant l'amplitude et la fréquence des rotations.

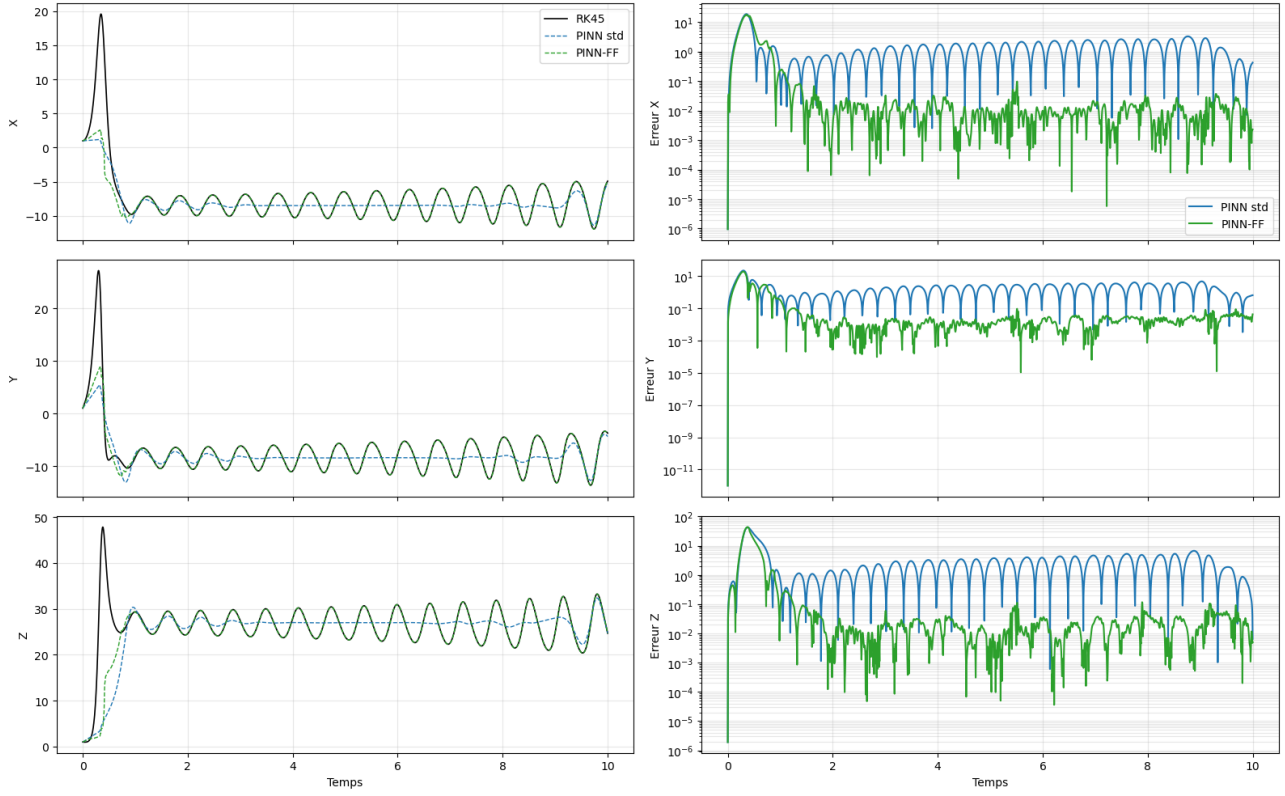


FIGURE 3.9 – Évolution temporelle des variables d'état (gauche) et erreurs de prédiction (droite, échelle logarithmique) pour le cas Y observé. Le PINN standard décroche dès $t \approx 0,5$ et se stabilise sur une valeur moyenne, incapable de suivre les oscillations. Le PINN-Fourier se superpose quasi parfaitement à la référence sur tout l'intervalle $[0, 10]$. Les erreurs relatives du modèle Fourier restent stables entre 10^{-2} et 10^{-3} , tandis que celles du PINN standard explosent dès le début pour se maintenir autour de 1.

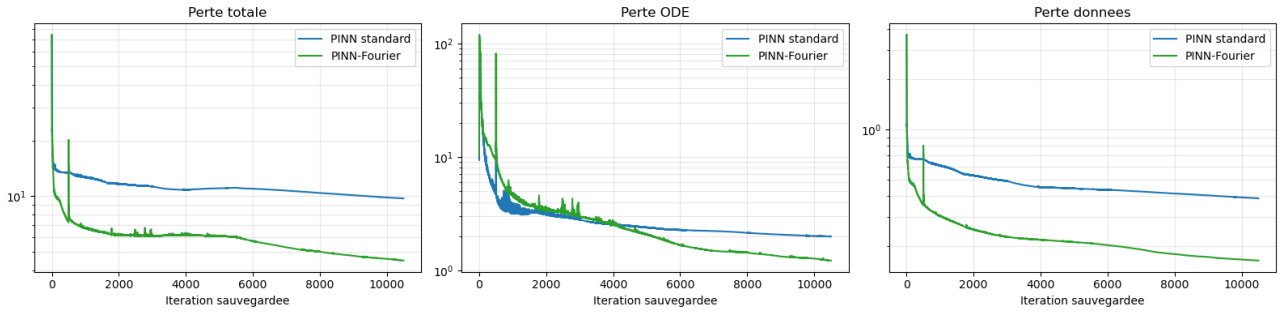


FIGURE 3.10 – Analyse des courbes d'apprentissage : pertes totale, ODE (physique) et données (cas Y observé). Le PINN-Fourier descend bien plus bas en perte totale après une chute initiale, tandis que le PINN standard plafonne rapidement. La perte ODE du modèle Fourier atteint environ 1), soit un ordre de grandeur en dessous du standard, prouvant une meilleure satisfaction des équations différentielles. La perte sur les données observées est également nettement inférieure pour le Fourier.

3.6.8 Interprétation (cas Y observé)

Les figures 3.8 et 3.9 montrent une nouvelle fois la supériorité du PINN-Fourier. Alors que le PINN standard s'effondre dès les premiers instants ($t \approx 0,5$) et ne peut reproduire

les oscillations, le modèle à caractéristiques de Fourier suit la référence avec une précision exceptionnelle, aussi bien dans l'espace des phases que temporellement. Les erreurs relatives du PINN-Fourier se maintiennent entre 10^{-2} et 10^{-3} , contre environ 1) pour le standard. L'étude des courbes d'apprentissage (figure 3.10) explique cet écart : le PINN-Fourier minimise beaucoup plus efficacement la perte ODE (jusqu'à 1)), ce qui signifie qu'il « comprend » mieux la physique du système de Lorenz. La perte de données est également réduite d'un ordre de grandeur. On note que, contrairement au cas X observé, il n'y a pas de pic d'instabilité marqué dans les courbes vertes, suggérant une convergence plus lisse pour ce cas.

3.6.9 Visualisations (cas X observé)

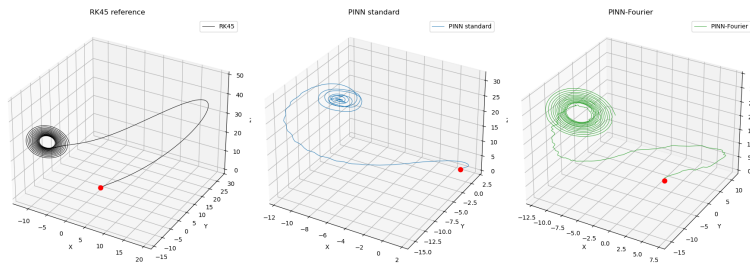


FIGURE 3.11 – Comparaison des trajectoires 3D et reconstruction de l'attracteur en n'observant que la variable X . La référence RK45 montre une structure en spirale convergeant vers un attracteur. Le PINN standard peine énormément : la trajectoire est bruitée, les spirales ne sont pas nettes et la forme globale est écrasée. Le PINN-Fourier, grâce aux caractéristiques de Fourier, capture bien mieux les hautes fréquences et les oscillations ; la structure en spirale est très proche de la référence, avec un léger bruit résiduel.

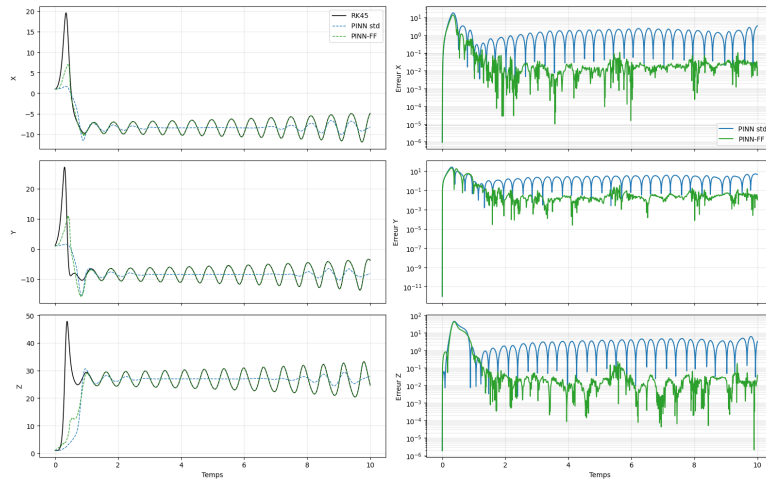


FIGURE 3.12 – Évolution temporelle des variables d'état (gauche) et erreurs de prédiction (droite, échelle logarithmique) pour le cas X observé. Le PINN standard échoue presque immédiatement après $t = 1$ et stagne sur une valeur moyenne ou une oscillation très amortie. Le PINN-Fourier suit la courbe de référence avec une précision impressionnante sur toute la durée. Les erreurs du PINN standard remontent rapidement pour se stabiliser entre 10^{-1} et 10) ; celles du PINN-Fourier restent globalement entre 10^{-2} et 10^{-3} , soit plusieurs ordres de grandeur plus bas.

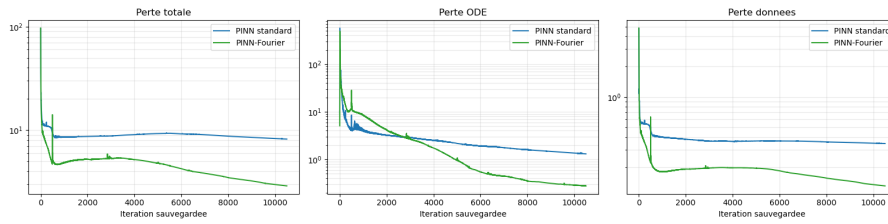


FIGURE 3.13 – Évolution des pertes au cours de l'entraînement (cas X observé). La perte totale du PINN-Fourier descend beaucoup plus bas que celle du PINN standard, qui plafonde autour de 1. La perte ODE (physique) est également beaucoup plus faible pour le modèle Fourier, indiquant une meilleure satisfaction des équations de Lorenz. On note un léger pic vers l'itération 500 sur les courbes vertes, probablement dû à un changement de régime d'apprentissage avant une convergence plus profonde.

3.6.10 Interprétation (cas X observé)

La figure 3.11 montre que le PINN-Fourier parvient à reconstruire la dynamique en spirale et l'attracteur global, alors que le PINN standard produit une trajectoire bruitée et déformée. L'analyse temporelle (figure 3.12) confirme l'échec précoce du PINN standard : après $t \approx 1$, ses variables s'écartent irrémédiablement de la référence. À l'inverse, le PINN-Fourier suit la référence avec une fidélité remarquable sur l'ensemble de la fenêtre temporelle, et les erreurs relatives restent très faibles (entre 10^{-2} et 10^{-3}) malgré quelques pics initiaux. Les courbes d'apprentissage (figure 3.13) expliquent cette différence : le PINN-Fourier minimise bien plus efficacement la perte ODE, ce qui signifie qu'il satisfait mieux les équations différentielles. Le plateau élevé du PINN standard sur la perte totale indique qu'il est bloqué dans un minimum local médiocre. Le pic observé vers l'itération 500 pour le modèle Fourier est typique d'une brève instabilité avant une convergence profonde.

3.6.11 Synthèse comparative des trois cas

Le tableau 3.5 rassemble les erreurs relatives L2 globales pour les trois configurations.

TABLE 3.5 – Erreurs relatives L2 globales pour les trois problèmes inverses.

Variable observée	PINN standard	PINN-Fourier	Meilleure architecture
Z seul	0.23516	0.20558	Fourier
Y seul	0.25135	0.19680	Fourier
X seul	0.27729	0.21517	Fourier

On observe que : - Le PINN-Fourier surpasse systématiquement le PINN standard, quel que soit le capteur utilisé. - La reconstruction à partir de Z donne la plus faible erreur globale pour le standard, tandis que Y donne la meilleure performance pour le Fourier. - La variable X semble la moins informative : l'erreur globale y est la plus élevée dans les deux architectures.

3.6.12 Conclusion partielle – problème inverse

La reconstruction à partir d'une seule variable est un problème mal posé pour le système de Lorenz : seules les variables observées sont correctement apprises, tandis que les variables cachées restent très incertaines. L'ajout de *Fourier Features* améliore la stabilité et la satisfaction des équations, mais ne peut compenser un manque d'observabilité. Pour des applications pratiques, il serait nécessaire d'utiliser plusieurs capteurs ou d'intégrer des techniques d'assimilation de données.

Synthèse, limites et perspectives

4.1 Bilan des résultats expérimentaux

L'étude du système de Lorenz a mis en évidence :

- la faiblesse des PINNs standards face aux dynamiques chaotiques et au contenu fréquentiel étendu ;
- l'apport décisif des Fourier Features, réduisant l'erreur et permettant de reconstruire correctement l'attracteur ;
- la faisabilité de la reconstruction à partir d'une seule variable, avec des performances variables selon la variable choisie (y donne les meilleurs résultats, z les moins bons) ;
- l'impossibilité d'obtenir une erreur nulle à long terme à cause du chaos, mais un meilleur conditionnement initial retarde la divergence.

4.2 Limites actuelles des PINNs

Malgré leurs succès, les PINNs souffrent encore de plusieurs limitations :

- **Sensibilité aux hyperparamètres** : le nombre de couches, de neurones, le choix des fonctions d'activation et la pondération des termes de perte influencent fortement la convergence.
- **Difficulté à capturer les hautes fréquences** : le biais spectral est atténué par les Fourier Features, mais pas complètement éliminé pour des fréquences très élevées ou des fronts raides.
- **Coût d'entraînement** : pour des problèmes en grande dimension ou des équations aux dérivées partielles complexes, l'entraînement peut devenir très long et nécessiter des ressources importantes.
- **Observabilité partielle** : la reconstruction à partir d'une seule variable montre que certaines composantes sont peu informatives, conduisant à des erreurs résiduelles élevées.

4.3 Perspectives

Plusieurs voies de recherche prometteuses se dessinent :

- **Architectures adaptatives** : utiliser des fréquences apprises (Fourier Features paramétriques) pour couvrir dynamiquement le spectre du problème.
- **Méthodes hybrides** : combiner PINNs et schémas numériques classiques (par exemple, utiliser un solveur RK pour les parties raides et un PINN pour les corrections).
- **Opérateurs neuronaux** : des architectures comme DeepONet ou les Neural Operators visent à apprendre directement l'opérateur solution d'une famille d'EDP, ouvrant la voie à des simulations en temps réel.
- **Assimilation de données** : exploiter les PINNs pour fusionner des mesures partielles avec un modèle physique, en particulier pour des systèmes où certaines variables ne sont pas accessibles expérimentalement.

Conclusion générale

Ce mémoire avait pour objectif d'évaluer la capacité des Physics-Informed Neural Networks à résoudre des équations différentielles ordinaires non linéaires, en les confrontant à une méthode numérique de référence, le solveur RK45. À travers l'étude approfondie du système chaotique de Lorenz, nous avons tiré plusieurs enseignements.

Tout d'abord, les PINNs constituent une approche flexible et prometteuse, particulièrement adaptée aux problèmes inverses et aux situations où les données sont rares. Leur capacité à fournir une solution continue et différentiable est un atout indéniable par rapport aux méthodes classiques. Cependant, leur performance dépend fortement de l'architecture du réseau et des hyperparamètres.

Le résultat le plus marquant de cette étude concerne le système de Lorenz. Le PINN standard, malgré une architecture de taille respectable, échoue à capturer correctement la dynamique chaotique et l'attracteur prédit est déformé. En revanche, l'ajout de **Fourier Features** réduit l'erreur d'un facteur important et permet de reconstituer l'attracteur avec une fidélité remarquable. Ce gain s'explique par la lutte efficace contre le **biais spectral** : les features sinusoïdales fournissent au réseau une représentation explicite des hautes fréquences, le déchargeant ainsi d'une partie difficile de l'apprentissage.

La reconstruction à partir d'une seule variable a montré que la variable y est la plus informative, suivie de x , tandis que z seul ne permet pas une reconstruction satisfaisante. Les Fourier Features améliorent nettement les performances dans tous les cas, mais ne peuvent compenser un manque d'observabilité intrinsèque.

En conclusion, les PINNs, et en particulier leur version enrichie par des features de Fourier, constituent une alternative crédible aux méthodes numériques classiques pour la résolution d'équations différentielles, à condition de maîtriser les hyperparamètres et d'adapter la représentation fréquentielle au problème. Les travaux futurs devraient explorer des fréquences apprises, des architectures hybrides et l'extension aux équations aux dérivées partielles en grande dimension.

Bibliographie

- [1] A. G. BAYDIN et al. “Automatic differentiation in machine learning : a survey”. In : *Journal of Machine Learning Research* 18.153 (2018), p. 1-43.
- [2] G. CYBENKO. “Approximation by superpositions of a sigmoidal function”. In : *Mathematics of Control, Signals and Systems* 2.4 (1989), p. 303-314.
- [3] I. GOODFELLOW, Y. BENGIO et A. COURVILLE. *Deep Learning*. MIT Press, 2016.
- [4] E. HAIRER, S. P. NØRSETT et G. WANNER. *Solving Ordinary Differential Equations I. Nonstiff Problems*. Springer, 2008.
- [5] K. HORNIK, M. STINCHCOMBE et H. WHITE. “Multilayer feedforward networks are universal approximators”. In : *Neural Networks* 2.5 (1989), p. 359-366.
- [6] D. P. KINGMA et J. BA. “Adam : A Method for Stochastic Optimization”. In : *arXiv preprint arXiv :1412.6980* (2014).
- [7] E. KREYSZIG. *Advanced Engineering Mathematics*. Wiley, 2011.
- [8] I. E. LAGARIS, A. LIKAS et D. I. FOTIADIS. “Artificial neural networks for solving ordinary and partial differential equations”. In : *IEEE Transactions on Neural Networks* 9.5 (1998), p. 987-1000.
- [9] E. N. LORENZ. “Deterministic nonperiodic flow”. In : *Journal of the Atmospheric Sciences* 20.2 (1963), p. 130-141.
- [10] D. C. PSICHOGIOS et L. H. UNGAR. “A hybrid neural network-first principles approach to process modeling”. In : *AIChE Journal* 38.10 (1992), p. 1499-1511.
- [11] M. RAISSI, P. PERDIKARIS et G. E. KARNIADAKIS. “Physics-informed neural networks : A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In : *Journal of Computational Physics* 378 (2019), p. 686-707.
- [12] S. H. STROGATZ. *Nonlinear Dynamics and Chaos*. Westview Press, 2015.

-
- [13] M. TANCİK et al. “Fourier features let networks learn high frequency functions in low dimensional domains”. In : *Advances in Neural Information Processing Systems* 33 (2020), p. 7537-7547.